科目：データベースの基礎

第8回 データベース管理システム

講師：鈴木源吾

お知らせ

- **小テスト2 は来週 11/25（第9回）**．範囲は第4〜7回，20分
  - 持ち込みは第4〜7回のポイント資料のみ（自分で記入した演習を含む）
  - 型F〜K（代数の結果，SELECT 文，SQL と代数の対応，ER 図，関数従属，正規化）．ポイント資料の例と演習を解き直しておくこと
- **第7回演習**の締切は今日の講義開始30分後．締切後も 2/19 18:30 まで提出できる（0.7点）
- **小テスト1 レポート**は締切後も 2/19 18:30 まで提出できる（満点6点）

お知らせ：第13回の日程変更（A クラスのみ）

- **A クラス（水曜2限）の第13回**は，12/23（水）2限が**休講**になり，**12/18（金）4限に補講**を行う
  - 第12回（12/16）の2日後．教室は決まり次第，WebClass で知らせる
  - B クラス（水曜1限）は変更なし．12/23（水）1限に第13回を行う
- 12/18（金）4限に出席できない事情がある人は，早めに申し出ること

お知らせ：学習支援センターの補習

- 学習支援センターが **毎週，小テストレポートの補習** を行う
  - **火曜4限**：留学生　／　**水曜3限**：留学生以外
  - 場所は原則 **S301**（**今日 11/18 は S308**）
- **対象**：小テストが **5点未満または欠席** で，締切を過ぎた演習の **過半数が未提出** の人
- **内容**：解答例を見ずに解き直し，質問しながらレポートを仕上げる．未提出の演習もその場で提出する
- 演習の未提出が多いと単位が危うくなる．対象の人は必ず参加すること

前回の復習

1. 正規化の背景と概要
    - 更新時異状（削除・挿入・修正）．2つの「もの」が1つのリレーションに混在するのが原因
2. 関数従属性と第2正規形
    - X → Y．主キーの一部で決まる（部分関数従属）→ 第2正規形に反する
3. 第3正規形
    - 非キー属性で決まる（推移的関数従属）→ 第3正規形に反する
4. 型J・K：関数従属を見つける，第2・第3正規形に分解する（決める側は残す，同じタプルは1つに）

第7回演習の解説

- 第7回ポイント資料の裏面（演習1・演習2・確認問題）の正解を解説する
- 解説は，演習の締切（講義開始30分後）のあとに行う
- 解答例は電子的には配布しない．**自分の解答に転記すること**（解説中の撮影は不可）
  - 講義中に自分の解答へ書き足す・訂正するのはよい．そのまま試験に持ち込める
- 提出がまだの人は，WebClass の「【締切後】第7回 演習」から提出する（0.7点で評価）
- 小テスト2（来週）の型J・K と同じ型なので，解き方を確認しておくこと

今回のトピック

1. DBMS の標準アーキテクチャ（3層スキーマ構造）
2. 外部スキーマの実現（ビュー）
3. 内部スキーマの実現（ファイル編成・インデックス・B+木）
4. 今回の演習と小テストの「型」：**インデックスで速くなるか・B+木の探索と挿入**

教科書：第9章 p.100〜114（9.1 DBMS の標準アーキテクチャ，9.2 DBMS の3層スキーマ構造，9.3 外部スキーマの実現―ビューサポート―，9.4 内部スキーマの実現―B+木―）

Topic 1

DBMS の標準アーキテクチャ

トピックのキーワード

- アーキテクチャ，ANSI/X3/SPARC
- 3層スキーマ構造（外部・概念・内部）
- データ独立性（物理的・論理的）

DBMS のアーキテクチャ

DBMS を「作る」とき，どのようなシステム構成にすべきか？

→ あるべきシステム構成：アーキテクチャ

- アーキテクチャという言葉は，建築物・設計・様式などの意味
  - 「コンピュータ・アーキテクチャ」はすでに学んだはず
- 「部品」の機能を明らかにし，部品どうしのインターフェースを決める
- すると部品の設計や交換が容易になる → 拡張しやすいなどの利点

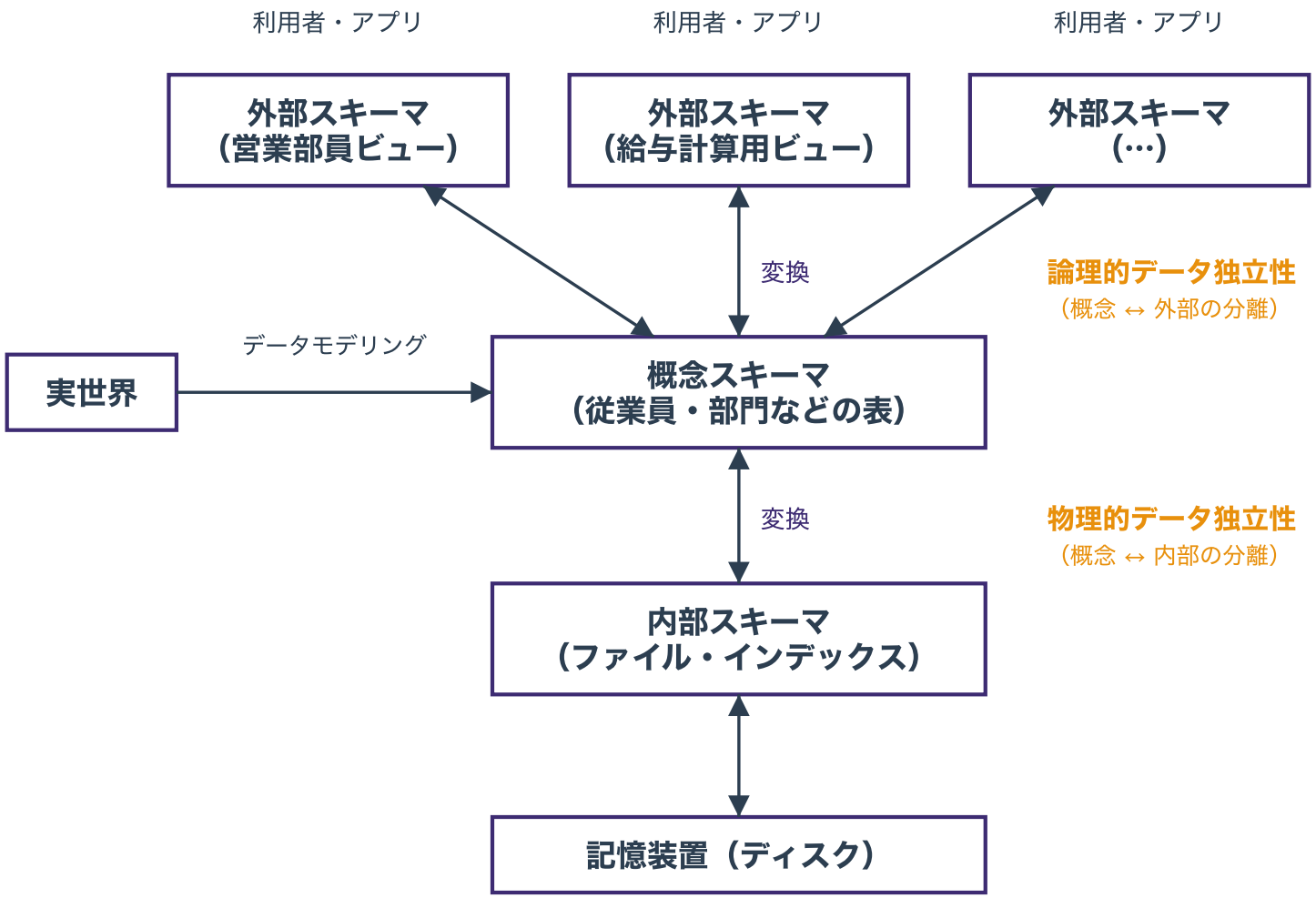
ANSI/X3/SPARC の3層アーキテクチャ

- DBMS の有名なアーキテクチャとして，米国規格協会（ANSI）の委員会が1970年代に提案した **ANSI/X3/SPARC のアーキテクチャ**がある
  - 「ANSI/SPARC の」と呼ばれることも多い
- 「役割を担った人」「処理機能」「インターフェース」「データ辞書」などを定めている
  - 全体図は教科書 9.1（p.100〜）を参照．とても複雑なので，本講義では要点だけを扱う
- 最大のポイントは，**スキーマを3層に分けて管理する**ことを提案したこと
  - **外部スキーマ・概念スキーマ・内部スキーマ**．これを DBMS の **3層スキーマ構造**と呼ぶ

参考：もう1つの「3層アーキテクチャ」

- Web システムの構成でも「3層アーキテクチャ」という言葉を使う．**今回の話とは別物**
  - プレゼンテーション層（ブラウザ・画面）→ アプリケーション層（Web サーバ上のプログラム）→ データ層（DBMS）
  - システムを役割で層に分ける話．DBMS はいちばん下の層にあたる
- ANSI/X3/SPARC の3層は，**DBMS の中で**スキーマを3つに分ける話
  - アプリケーション層のプログラムが SQL で見ているのは，外部スキーマ（ビュー）や概念スキーマ（表）
- ANSI/X3/SPARC の提案は1970年代のもので，その構成どおりの製品があるわけではない
  - ただし，スキーマを分ける考え方は，今の DBMS のビュー・インデックスに生きている

3層スキーマ構造


利用者・アプリ
利用者・アプリ
利用者・アプリ
外部スキーマ
（営業部員ビュー）
外部スキーマ
（給与計算用ビュー）
外部スキーマ
（…）
変換
論理的データ独立性
（概念 ↔ 外部の分離）
データモデリング
実世界
概念スキーマ
（従業員・部門などの表）
変換
物理的データ独立性
（概念 ↔ 内部の分離）
内部スキーマ
（ファイル・インデックス）
記憶装置（ディスク）

3層スキーマ：概念スキーマ

- 実世界を，DBMS の持つデータモデルによって記述したもの
  - 本講義のリレーショナル DBMS では，表（リレーション）・属性・制約の集まり．データベースに1つ
  - データベース設計者が実世界をどう認識したかの表現（第6・7回の設計の結果）
- 「どう格納しようか」「利用者の見え方をどうしようか」は，別のスキーマで考える
- 用語の注意：第6回では，ER 図で描いた概念設計の結果を「概念モデル（概念スキーマ）」と呼んだ．3層スキーマの概念スキーマとは，同じ用語だが意味が違う

3層スキーマ：内部スキーマと外部スキーマ

内部スキーマ：概念スキーマを，実際のコンピュータ上に格納するためのスキーマ

- リレーションをどうファイルに格納するか（ファイル編成）
- 速く探すためにどんなインデックスを付けるか（Topic 3）

外部スキーマ：概念スキーマを，**利用者・アプリケーションごと**に都合のよい形にしたもの

- 概念スキーマはアプリと独立に設計するので，各アプリにとって使いやすいとは限らない
- いくつあってもよい．リレーショナル DBMS では **ビュー** で実現する（Topic 2）

3層スキーマと管理者

3層アーキテクチャでは，スキーマごとに責任を負う人（役割）も決まっている

1. **組織体管理者**（enterprise administrator）
    - 概念スキーマを定義する．データ管理者と呼ばれることもある
2. **データベース管理者**（database administrator）
    - 概念スキーマを内部スキーマに変換する（格納方法・インデックスを決める）
3. **アプリケーションシステム管理者**（application system administrator）
    - 概念スキーマをもとに，さまざまな外部スキーマを定義する

3層スキーマの意義：データ独立性

データ独立性：アプリケーションが，データの細かい格納方法（レコードの格納順，インデックスの有無など）を意識しなくてよいこと．3層に分けることで，次の2つが得られる

	物理的データ独立性	論理的データ独立性
分離するもの	概念スキーマ と **内部**スキーマ	概念スキーマ と **外部**スキーマ
変えてもアプリはそのまま	ファイルの形式，インデックスの有無	表の構造（ビューの定義は直す）

- 例：インデックスを追加しても，SELECT 文は1文字も変えなくてよい（物理的）

Topic 2

外部スキーマの実現（ビュー）

トピックのキーワード

- ビュー（仮想的な表）
- CREATE VIEW

ビュー（第5回の復習）

- アプリケーションはテーブルの一部しか使わない → 最初からテーブルの一部だけを見せる
- このような「仮想的な」表を**ビュー**と呼ぶ．ビューは問合せ（SELECT 文）で定義する

```
CREATE VIEW 営業部員 AS
  SELECT 氏名, 住所 FROM 従業員 WHERE 部門番号 = 1;

SELECT 氏名 FROM 営業部員;    -- 佐藤一郎，田中次郎
```

- ビューへの問合せは，ビュー定義の問合せと合成したものが実行される
  - `SELECT 氏名 FROM 従業員 WHERE 部門番号 = 1;` と同じ結果
- 第4回の環境（pg18）の sample で実際に作って試せる

ビューのイメージ

従業員（実際にある表）

従業員番号	部門番号	氏名	住所	職級
1	1	佐藤一郎	横浜市	2
2	1	田中次郎	静岡市	1
3	2	鈴木三郎	横浜市	2
4	2	山田四郎	東京都	1

部門番号 = 1 の行（WHERE）

氏名・住所の列（SELECT）



営業部員（ビュー）

氏名	住所
佐藤一郎	横浜市
田中次郎	静岡市

データは持たない（仮想的な表）

- ビューはデータを持たない．元の表（従業員）が更新されれば，ビューの結果も変わる
- ビューに対する行の追加・変更・削除には制限がつくこともある（元の表のどの行か決まらない場合など）

ビューと論理的データ独立性

- 例：従業員の表を，従業員（従業員番号, 部門番号, 氏名, 職級）と 住所録（従業員番号, 住所）に分割した
- 営業部員を使うアプリは，そのままでは動かなくなる（従業員に 住所 がない）
- ビューの定義だけを直せば，アプリは変更しなくてよい

```
CREATE VIEW 営業部員 AS
  SELECT 氏名, 住所 FROM 従業員 JOIN 住所録
    ON 従業員.従業員番号 = 住所録.従業員番号
  WHERE 部門番号 = 1;
```

- これが**論理的データ独立性**（概念スキーマが変わっても，外部スキーマでその違いを吸収する）

Topic 3

内部スキーマの実現

トピックのキーワード

- ファイル編成，レコード，ブロック
- インデックス，ポインタ
- 多段インデックス，B+木

ファイル編成とは？

DBMS がデータを格納するには，次の2つの世界を対応づける必要がある

- DBMS の世界で見えるデータ：テーブル・タプル
- OS の世界で見えるデータ：ファイル・ブロック（読み書きの単位．ページとも呼ぶ）

タプルを内部で表したものを**レコード**と呼ぶ．ファイルには複数のレコードが入る

ファイル・ブロックとレコードの対応のさせ方を**ファイル編成**と呼ぶ

→ **ファイル編成によって，ディスクの読み書きの回数が変わる → 性能が変わる**

リレーションの基本的な格納方法

最も基本的なのは

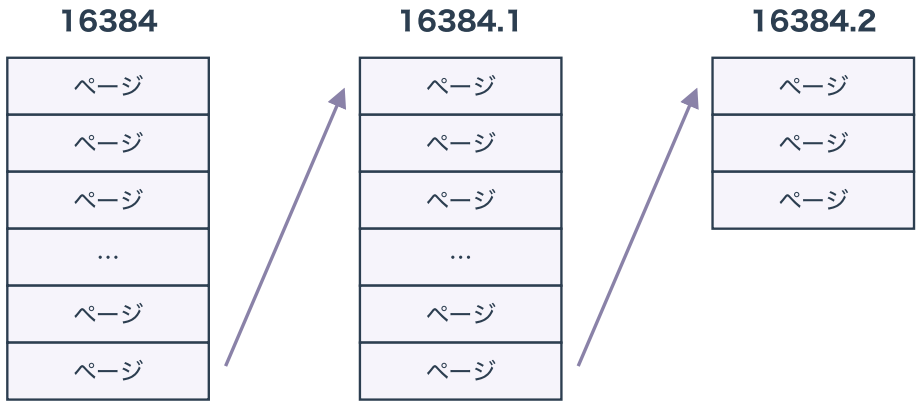
- 1リレーション → 1ファイル
- 1タプル → 1レコード

とする方法．1つのファイルに，1つのリレーションのすべてのタプルが入る

- 順序をつけずに並べる編成を**ヒープ**と呼ぶ（追加は末尾に書くだけで速い）
- 実際の DBMS は，巨大なテーブルを複数のファイルに分けるなどの工夫をしている（次のページ）

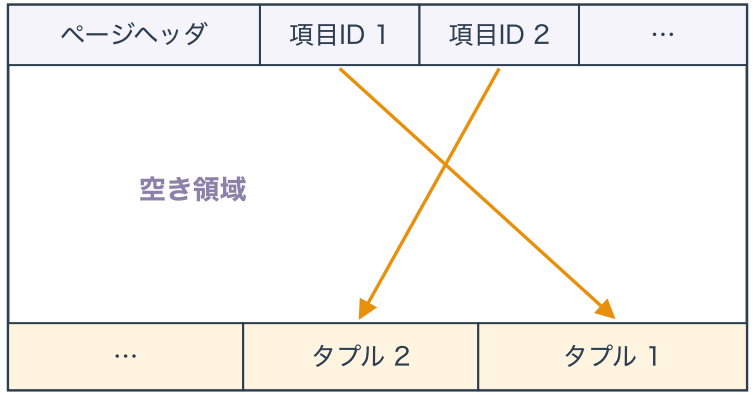
参考：PostgreSQL のファイル編成

ファイルの構成（1つのテーブル）



1ページ=8KB（ブロック）．1ファイルは最大 1GB（131,072ページ）
1GB を超えると，続きを別のファイル（.1，.2，…）に入れる
ファイル名の数字はテーブルごとに決まる番号（例）

ページ内のレイアウト（1ページ）



項目ID（タプルの位置と長さ）は前から，タプル（レコード）は後ろから詰める
両側から空き領域に向かって増えていく

- ページの中にタプルが入る．DBMS はページ単位でディスクを読み書きする（マニュアル「データベースの物理格納」）
- pg18 で `SELECT pg_relation_filepath('従業員');` を実行すると，ファイルの場所が分かる

インデックス

- レコードの**キー値**と，その**格納番地（ポインタ）の対応表をインデックス**と呼ぶ
- キー値の順に並べておく → 2分探索などですぐ見つかる．レコードを全部読まなくてよい

WHERE 社員番号 = '1508' → インデックスで番地 0002 が分かり，そのレコードだけ読む

インデックス（社員番号）

社員番号	格納番地
0231	0003
0650	0001
1508	0002
2034	0004
2100	0005

キー値の順に並ぶ（探しやすい）

データファイル（社員）

番地	社員番号	社員名	給与	所属
0001	0650	山田太郎	50	K55
0002	1508	鈴木花子	40	K41
0003	0231	田中桃子	60	K41
0004	2034	佐藤一郎	40	K55
0005	2100	森虎丸	40	K58

レコードは順不同（ヒープ）

インデックスで速くなる問合せ

- インデックスで速くなるのは，**インデックスを張った（作った）属性を WHERE の条件に指定した問合せだけ**
  - 前のページの例では 社員番号 を条件に指定したとき．すべての問合せが速くなるわけではない
- 1つのテーブルに複数のインデックスを張ってよい．複数の属性の組合せに1つ張ることもできる
- インデックスで探す方法を**インデックススキャン**，全レコードを読む方法を**全表走査**と呼ぶ
- 注意：データを追加・削除するたびにインデックスも直すので，更新は遅くなる

SQL でインデックスを作る

```
CREATE INDEX 従業員_住所_idx ON 従業員 (住所);    -- 住所 にインデックスを張る
DROP INDEX 従業員_住所_idx;                        -- 削除
```

- インデックスの名前は自由に付けてよい（表名_属性名_idx などが多い）
- 主キー（PRIMARY KEY）には，DBMS が自動でインデックスを作る（PostgreSQL の場合）
- インデックスを張っても，SELECT 文は変わらない（**物理的データ独立性**）．使うかどうかは DBMS が決める（第9回）
- PostgreSQL の CREATE INDEX の既定は B-tree（中身は次に学ぶ B+木の一種）

多段インデックス

インデックスには，次の要求がある

- 小さくしたい（メモリに載りやすくする）
- データの追加・削除のたびに直す手間を，できるだけ小さくしたい（全体を並べ直すのは大変）

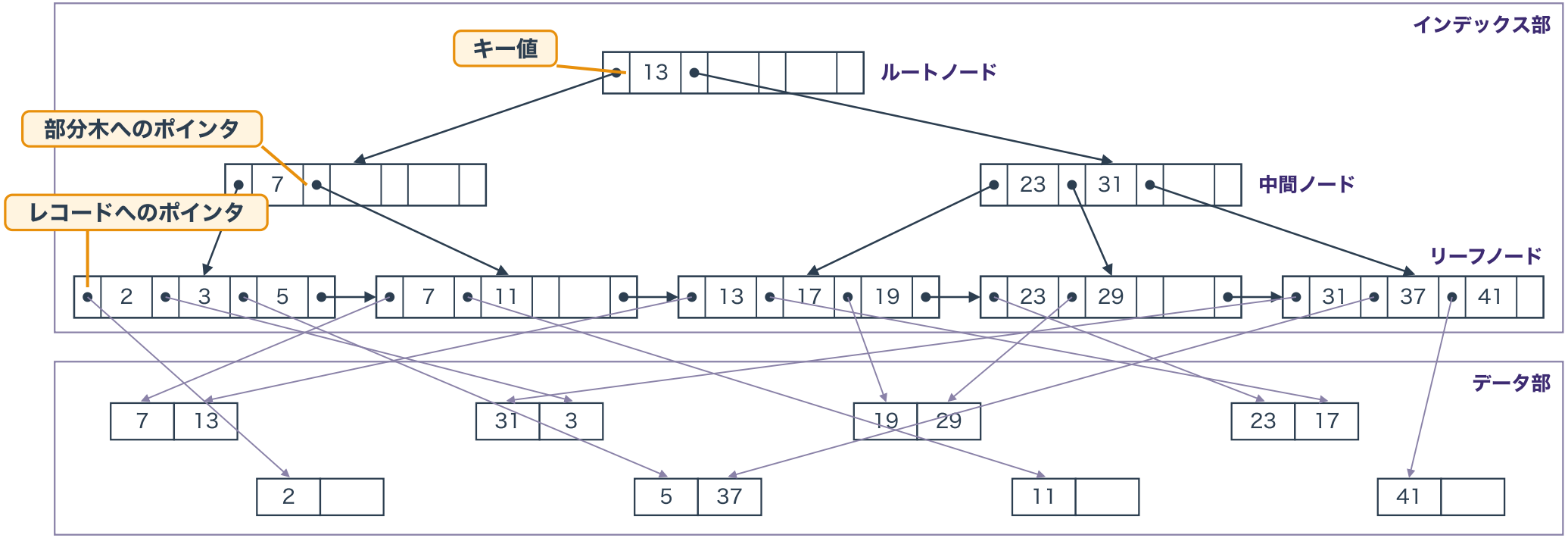
この要求に応えるのが**多段インデックス**

- インデックスを階層にして，木（tree）の形にする
- 大きなデータでも少ない段数で探せる．データの追加・削除に強い
- 例：**B+木**（最も普及している），B 木（B+木のもとになった），ISAM（かつてよく使われた）

B+木（B+-tree）

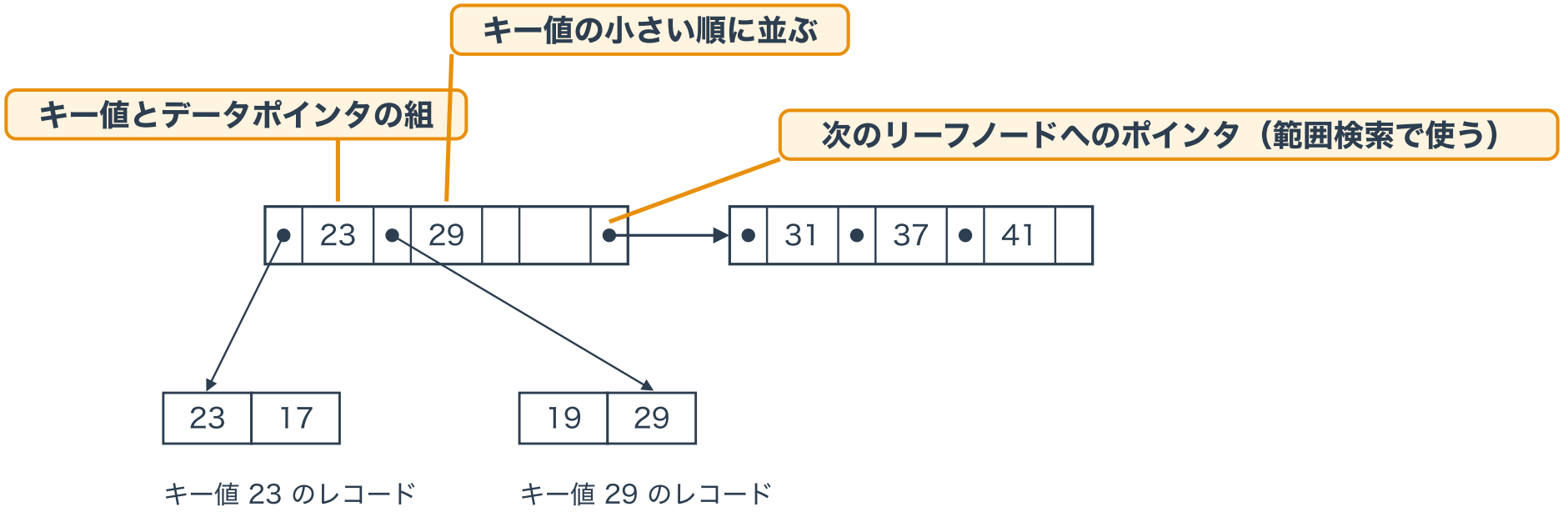
- 木は，複数のノードを階層に並べた構造（各ノードの親は1つ）
  - 最上位のノードを**ルートノード**（根．1つ），最下位のノードを**リーフノード**（葉），その間を**中間ノード**と呼ぶ
- ルートノードから各リーフノードまでの段数は，すべて同じ
  - このような木を**バランス木**（平衡木）と呼ぶ．B の由来には諸説ある（balanced，考案者の Bayer など）
- データが増えると木の段数が増える．追加・削除があっても，バランス木が保たれるように処理する
- 1ノードに入るキー値の最大個数を，本講義では**オーダ**と呼ぶ（以下の例はオーダ3）
  - オーダの定義は本によって違う（最大のポインタの個数とする本もある）

B+木の全体像



- データ部のレコードはヒープ（キー値の順に並んでいない）．鈴木・藤塚「RDBMS 解剖学」p.58 を参考に作成

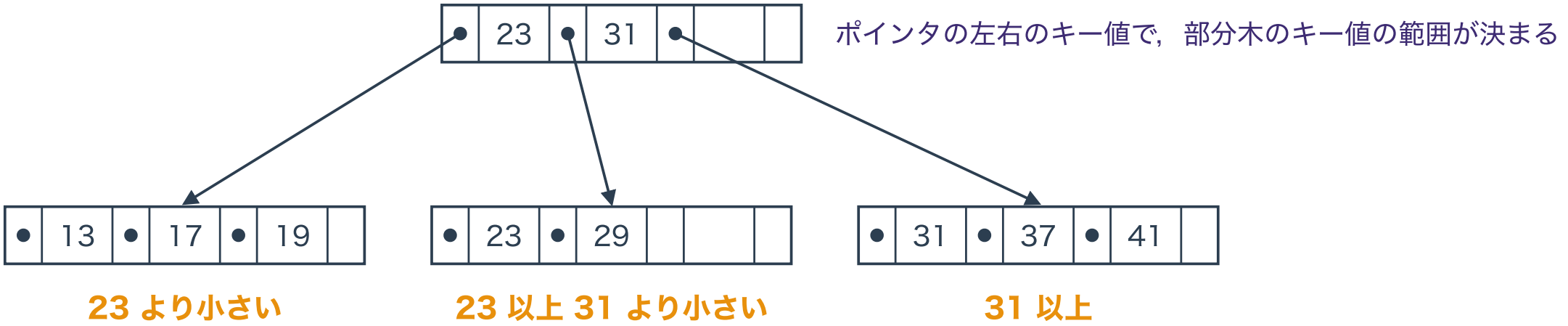
B+木の構造：リーフノード



1ノードのキー値の最大個数をオーダと呼ぶ（この例はオーダ3）．ノードは約半分以上が埋まるように保たれる

- すべてのキー値が，どれかのリーフノードに入っている
- 最後のポインタで次のリーフにつながる → 「29 以上 40 以下」のような範囲検索は，リーフを右へたどればよい

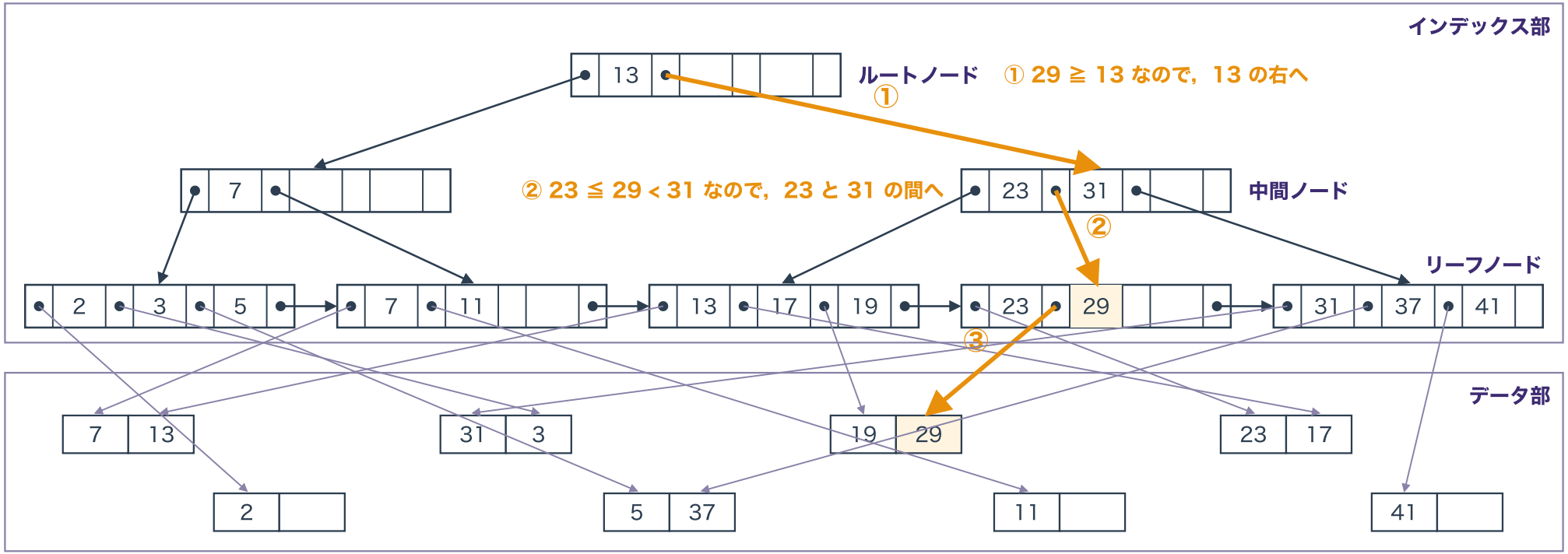
B+木の構造：中間ノード・ルートノード



- キー値 k の**左のポインタは k より小さい**キー値，**右のポインタは k 以上**のキー値の部分木を指す
  - 23 と 31 の間のポインタ：23 以上 31 より小さい
- 中間ノード・ルートノードのキー値は「道案内」．レコードへのポインタはリーフにだけある

B+木を使ったレコード探索

問合せ：キー値が 29 であるレコードを求めよ



- ③ リーフで 29 が見つかればレコードへ．読むノードは段数ぶん（3つ）だけ．100万件でも数段で済む

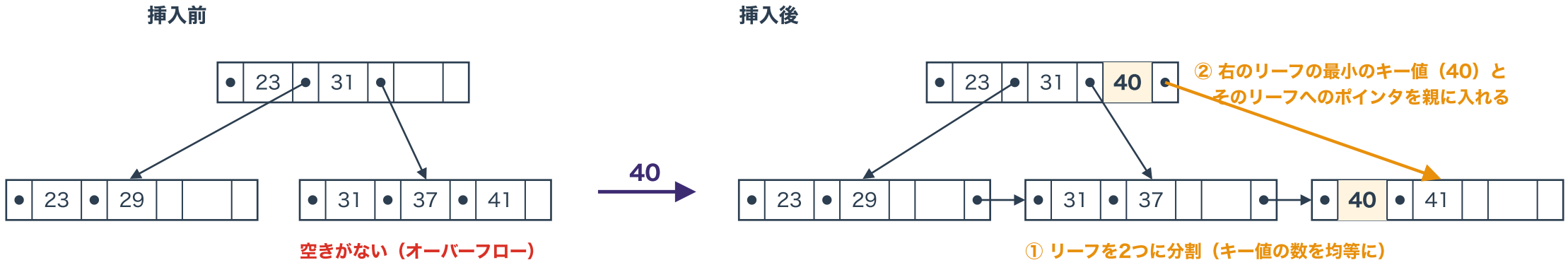
B+木の維持：レコードの挿入（空きがある場合）



空きがあれば，キー値の順の位置にキー値とレコードへのポインタを入れるだけ．中間ノードは変わらない

- 挿入するキー値が入るべきリーフノード（探索と同じ方法で見つける）に空きがあれば，キー値とレコードへのポインタを順序どおりの位置に入れるだけ
- 中間ノードの変更は不要

B+木の維持：レコードの挿入（空きがない場合）



1. リーフノードに空きがない（入れるとオーバーフロー）→ **リーフノードを2つに分割**する（キー値の数を均等に：31, 37, 40, 41 → [31, 37] と [40, 41]）
2. **右のリーフの最小のキー値**（40）と，そのリーフへのポインタを親（中間ノード）に入れる

分割は上へ伝わることがある

- 前の例では，運よく親の中間ノードに空きがあった
- 親にも空きがなければ，親の中間ノードも分割し，その影響がさらに上のノードに及ぶ（同じ処理の繰り返し．再帰的）
- ルートノードがあふれると，ルートノードが分割されて新しいルートができ，**木の段数が1つ増える**
  - だから，どのリーフまでの段数も同じまま（バランス木）が保たれる

B+木の維持：レコードの削除

- あるキー値のレコードが削除されたら，そのキー値の入ったリーフノードから，キー値とポインタを削除する
- それによって，リーフノードのキー値が少なくなりすぎた場合（オーダの約半分より少ない）を**アンダーフロー**と呼ぶ
- アンダーフローが起きたら，左右の兄弟ノードからキー値とポインタを分けてもらう（再配分）
- 再配分でも解消できないときは，兄弟ノードと合体する
- 詳しい条件は教科書 9.4 を参照（小テストでは削除は出さない）

Topic 4

今回の演習と小テストの「型」

型L：インデックスで高速化できるか（○×）

- **インデックスを張った属性が WHERE の条件にあれば○**
  - =，<，>= など範囲の条件でもよい（B+木ならリーフをたどる）．AND でつないだ条件の1つでもよい
- 条件がない（全件を読む），または条件の属性にインデックスがなければ×

例：社員（インデックスは 所属 だけ．`...` は `SELECT 社員名 FROM 社員`）

問合せ	答	理由
`... WHERE 所属 = 'K41';`	○	所属 の条件
`... WHERE 給与 = 40;`	×	給与 にインデックスがない
`SELECT * FROM 社員;`	×	全件を読む
`... WHERE 給与 = 40 AND 所属 = 'K55';`	○	所属 で絞ってから給与を調べる

型M：B+木の探索・挿入

- **探索**：参照する中間ノード・リーフノードを，ノードの最初のキー値で答える（例：29 → 中間ノード（23）→ リーフノード（23））
  - 各ノードで「k より小さければ左，k 以上なら右」．**等しいときは右**
- **挿入**：空きがなければ，リーフを2つに分割（キー値の数を均等に）．**右のリーフの最小のキー値**を親に入れる
- 例はポイント資料の表面（キー値 29 の探索，キー値 40 の挿入）．スライドと同じ木

今回の小テストの「型」（小テスト3：12/16）

- **型A：説明→用語**（第1回と同じ型）
  - 3層スキーマ（外部・概念・内部），データ独立性（物理的・論理的），ビュー
- **型L：インデックスで高速化できるか（○×）**
- **型M：B+木の探索・挿入**
- 小テスト3 の範囲は第8〜10回．持ち込みは第8〜10回のポイント資料
- その前に，来週 11/25 は**小テスト2**（範囲は第4〜7回）

第8回演習と次回

- ポイント資料の裏面を，表面の例にならって自分の手で解く
  - 演習1：型M の類題（キー値 37・13・7 で参照するノード）
  - 演習2：型M の類題（キー値 4 の挿入による分割）
  - 演習3：型L の類題（学籍簿のインデックス）
  - 確認問題：3層スキーマの用語（型A）
- 提出：表面・裏面を撮影して WebClass「第8回 演習」へ．締切：**次回（11/25）の講義開始30分後**
- 次回 11/25：**小テスト2**（第4〜7回）と，第9回 質問処理の最適化．予習：教科書 第10章 p.115〜127